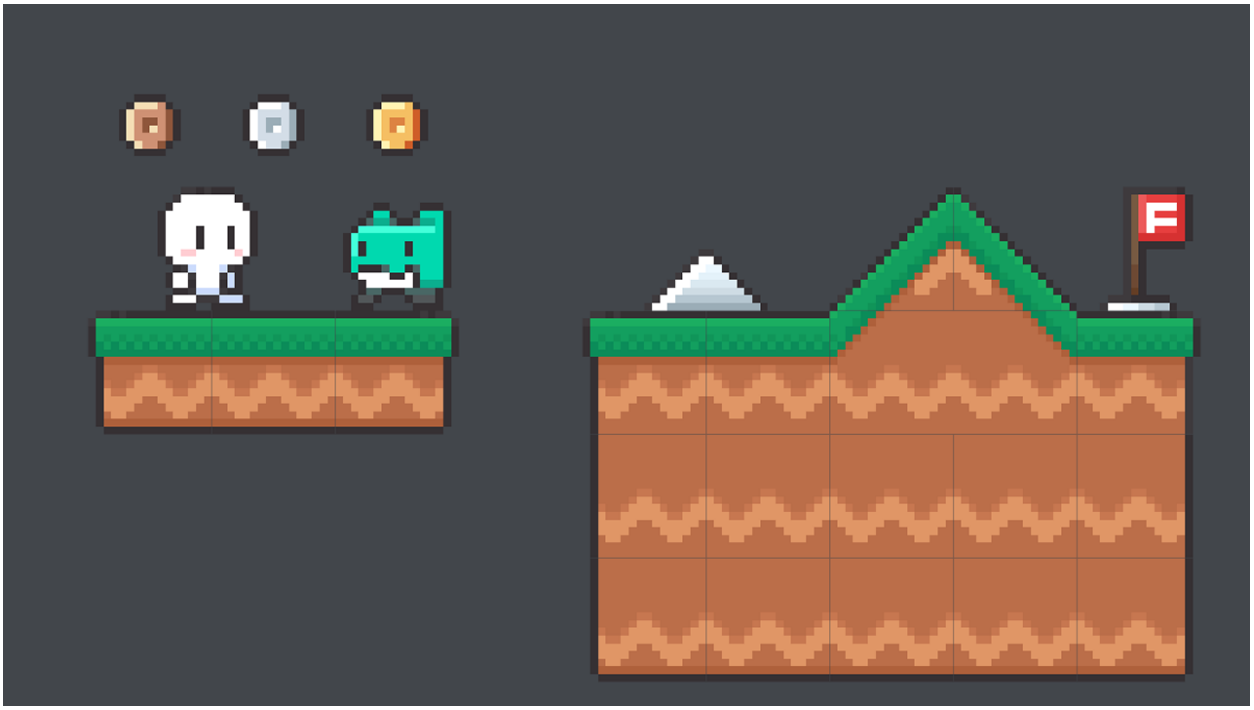
SLIIT UNI

THE KNOWLEDGE UNIVERSITY

SE4031 - Games Development

Assignment 1 – 2D Game

Spook Sprint - Game Documentation



IT22599704 – Samarasekara DP

Contents

1. Summary of the Game Concept

Spook Sprint is a 2D side-scrolling platformer built in Unity 2022.3. The player controls Boo - a cute white ghost character - navigating a grassy platformer world filled with metallic spike obstacles, wandering green enemies called Grumps, and collectable coins. Unlike typical auto-runners where the world scrolls automatically, the player has full directional control - moving left, right, jumping, and shooting.

The goal is to jump across platforms, defeat or avoid enemies, collect health packs to survive, and reach the red flag finish line. The game progressively increases in difficulty as the player's movement speed increases over time and enemies spawn more frequently and walk faster. The game ends either when the player's health reaches zero or when they successfully reach the finish line.

The game stands out from typical student submissions through its unique combination of free player movement (not auto-run), platform-gap based scoring, stomp-and-shoot enemy mechanics, and a polished visual feedback system featuring screen shake and red flash on damage - using a custom AI-generated asset set that gives the game a distinct visual identity.

Detail	Value
Game Title	Spook Sprint
Engine	Unity 2022.3.62f3
Genre	2D Side-Scrolling Platformer
Platform	PC (Windows)
Asset Pack	Goldmetal Simple 2D Platformer Assets Pack (Free - Unity Asset Store)
Custom Assets	Health pack, projectile, Game Over screen, Level Complete screen (AI-generated) with Custom AI-Generated Assets
Module	SE4031 Games Development

2. Description of Core Features

2.1 Health System

The player character Boo has a maximum of 100 HP displayed as a green UI slider bar in the top-left corner of the screen. Health decreases by 25 points upon collision with spike obstacles or enemy side-contact. The player passes through obstacles after taking damage - they are not stopped by the obstacle itself. When health reaches 0, the player's movement is disabled and the Game Over screen appears. Health resets to 100 when the game is restarted via the R key.

Implementation: PlayerHealth.cs handles all health logic. A damage cooldown of 0.5 seconds prevents multiple hits in quick succession. The UI Slider value is updated as a ratio of currentHealth / maxHealth on every damage or heal event.

2.2 Score System

The score is displayed as a TextMeshPro UI element in the top-right corner. Score increments in two ways: +1 when the player successfully jumps from one platform and lands on the next (the gap between platforms represents the hole in the obstacle), and +2 when the player destroys an enemy using the spirit orb projectile. Score resets to 0 when the game is restarted.

Implementation: ScoreManager.cs uses a static instance pattern so any script can call ScoreManager.instance.AddScore(amount) from anywhere. PlatformLanding.cs on each platform detects when the player lands on top and awards +1 score on first contact per platform. Enemy.cs awards +2 score when the Die(true) method is called by a projectile hit.

2.3 Health Packs

Health packs are orange rectangular collectables that restore 25 HP when the player runs into them, up to the maximum of 100 HP. They use a custom AI-generated heart sprite to make them visually distinctive. Health packs are spawned randomly alongside enemies and auto-destroy after 8 seconds if the player does not collect them. They are visually distinct from enemies - different shape, color, and sprite.

Implementation: HealthPack.cs uses Destroy(gameObject, 8f) in Start() for the auto-destroy timer. OnTriggerEnter2D detects Player tag collision and calls PlayerHealth.Heal(25). The collider is set to Is Trigger so the player passes through rather than being blocked.

2.4 Player Projectile Attacks

The player fires a spirit orb projectile on left mouse click. The orb is created at the player's position and travels toward the mouse cursor position in world space. It destroys on contact with an enemy and automatically self-destructs after 2 seconds. A 0.3 second cooldown prevents rapid fire spam. The projectile uses a custom AI-generated glowing orb sprite.

Implementation: Shoot.cs calculates the direction vector from the player position to the mouse world position using Camera.main.ScreenToWorldPoint(). SpiritOrb.cs stores this direction and applies it in Update() using transform.Translate(). OnTriggerEnter2D on the orb calls enemy.Die(true) when it hits an enemy tag.

2.5 Enemies

Teal green Grump creatures spawn from off the right edge of the screen at random platform heights and walk horizontally toward the player. Enemies are collidable objects that interact with the player in two ways: a stomp from above destroys the enemy with no damage to the player, while a side collision destroys the enemy and deals 25 damage to the player. Enemies are also destroyed instantly by the spirit orb projectile. Enemies auto-destroy after 15 seconds if the player goes past them. The enemy is visually distinguishable from health packs through different color, shape, and sprite.

Implementation: Enemy.cs uses a Rigidbody2D for physics. In OnCollisionEnter2D, the contact normal Y value determines stomp vs side hit - normal.y < -0.5f means the player hit from above. The walkSpeed field is set by EnemySpawner.cs and increases over time.

2.6 Speed Increase Over Time

The player's movement speed starts at 5 units per second and gradually increases by 0.02 per second, capped at a maximum of 10. This makes the game progressively harder as the player must time jumps more precisely at higher speeds. Additionally, enemies spawn more frequently over time (spawn interval decreases from 5 seconds to a minimum of 2 seconds) and each new enemy walks faster than the last (enemy walk speed increases from 2 to a cap of 6). These three compounding systems create a natural difficulty curve.

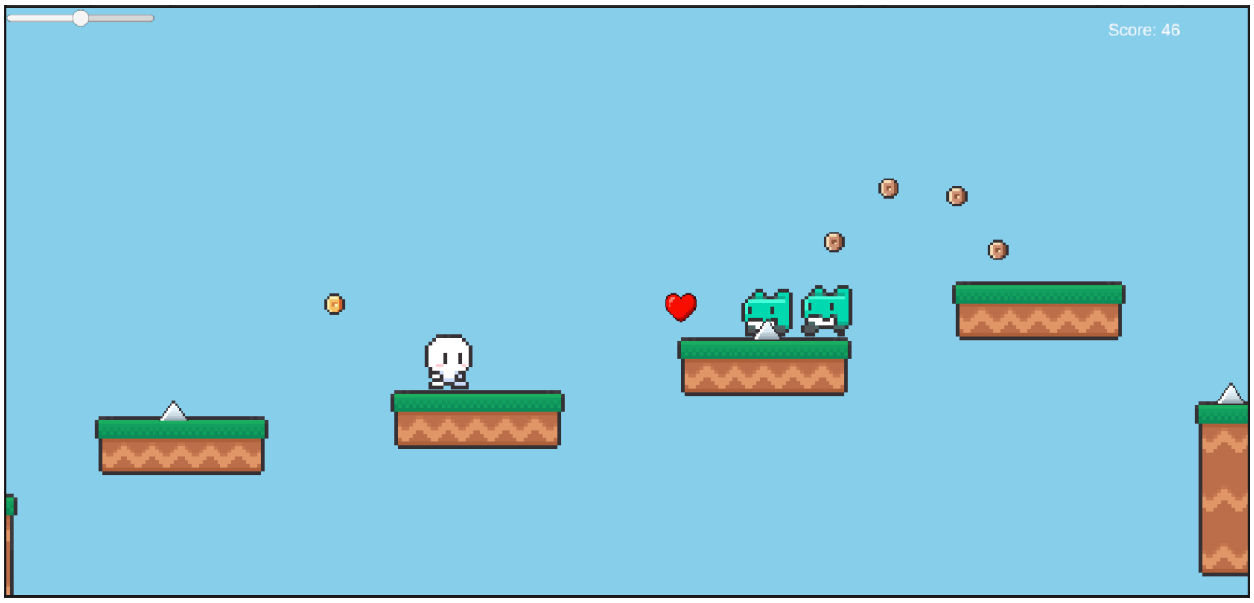
Implementation: PlayerMovement.cs handles player speed scaling in Update(). EnemySpawner.cs handles both the spawn interval decrease and the enemyBaseSpeed increase, passing the current speed to each newly spawned enemy via enemy.walkSpeed = enemyBaseSpeed.

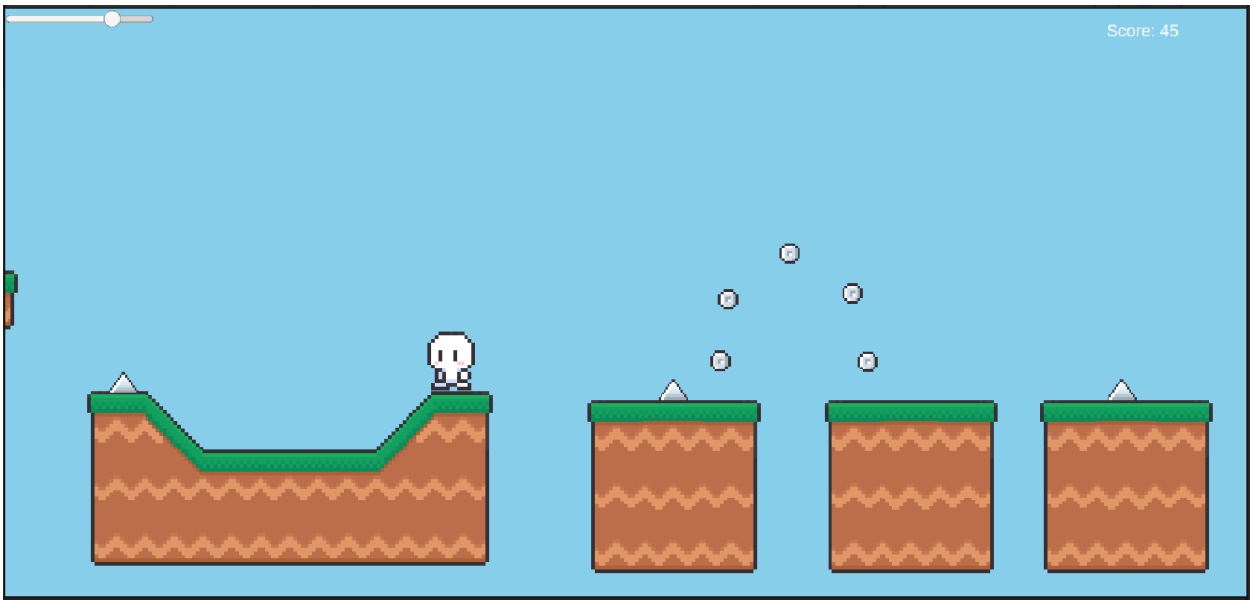
2.7 Creative Feature - Screen Shake & Red Flash

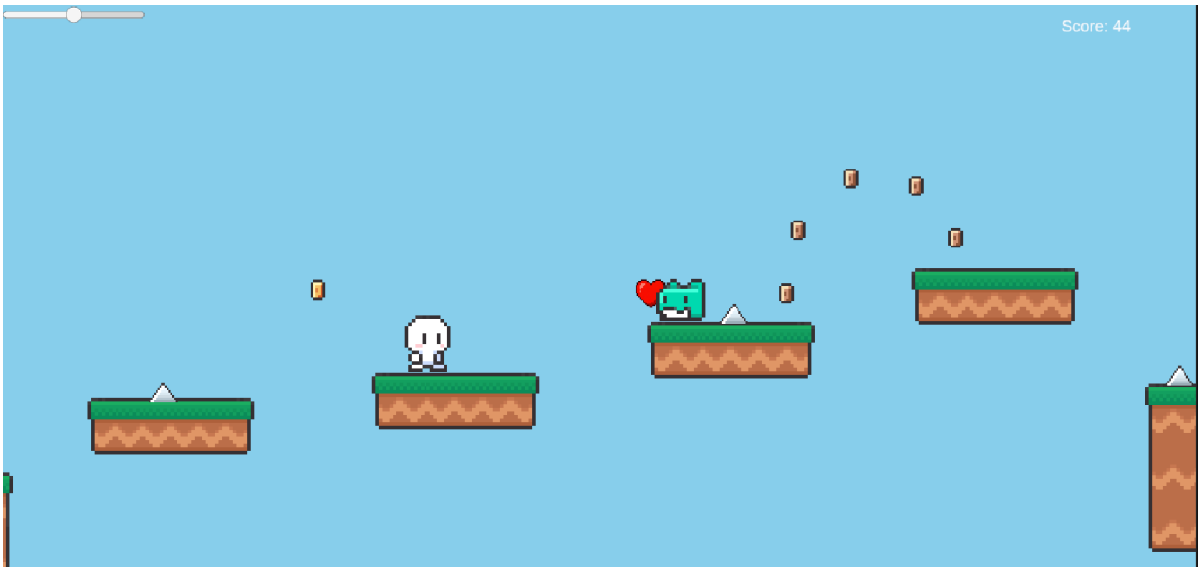
Every time the player takes damage from any source (spike, enemy side collision, or falling into a gap), two visual effects trigger simultaneously. The Main Camera shakes for 0.3 seconds using a coroutine that applies random position offsets each frame. At the same time a full-screen semi-transparent red UI Image overlay fades in instantly and fades out over 0.4 seconds. Together these effects make every hit feel impactful and provide clear visual feedback to the player. The combined shake-plus-flash effect is unique and directly referenced as a suggested creative feature in the assignment brief.

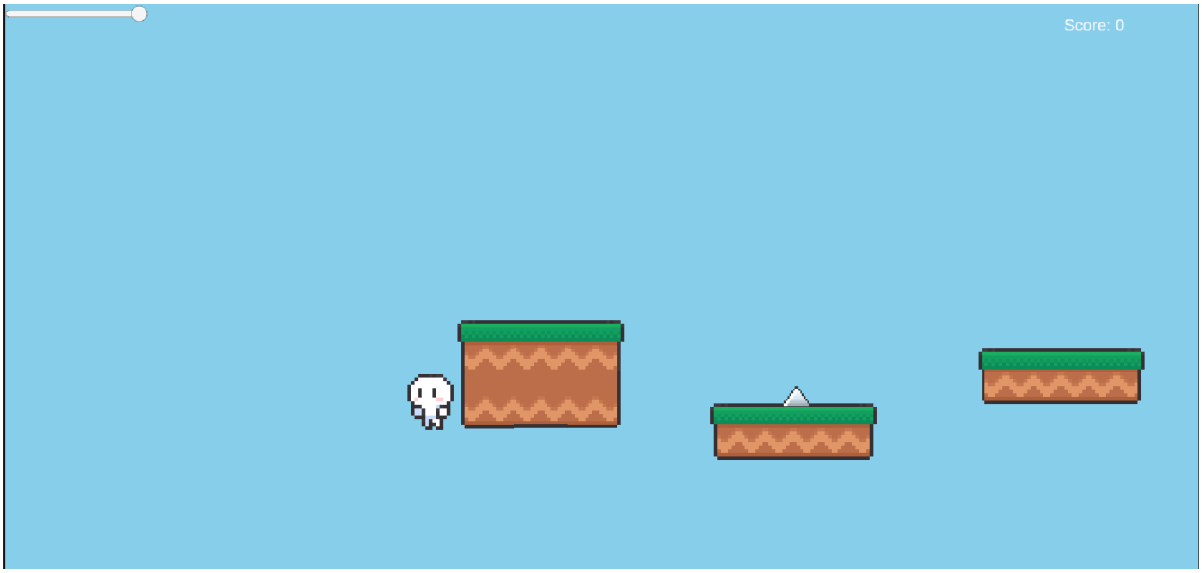
Implementation: CameraShake.cs is attached to the Main Camera and uses a coroutine with Random.insideUnitCircle * shakeMagnitude applied to the camera's local position each frame. DamageFlash.cs is attached to the Canvas and uses a coroutine that lerps the CanvasGroup alpha from 0.35 to 0 over the flash duration. PlayerHealth.cs calls both cameraShake.Shake() and damageFlash.Flash() whenever TakeDamage() is invoked.

3. Screenshots of Gameplay






Score: 44


Score: 0


Score: 87
GAME OVER
TRY AGAIN?
PRESS R

4. Control Guide

Input	Action
A / Left Arrow	Move player left
D / Right Arrow	Move player right
Space / Up Arrow	Jump (only when standing on ground or platform)
Left Mouse Click	Fire spirit orb projectile toward mouse cursor position
R	Restart the game after Game Over or Level Complete

5. Creative Feature Description

Screen Shake + Red Flash on Damage

The creative feature implemented in Spook Sprint is a dual visual feedback system that triggers every time the player takes damage from any source - spikes, enemy collisions, or falling into gaps.

The first effect is a camera shake: the Main Camera's position is randomized each frame within a small radius (0.15 units) for 0.3 seconds using a Unity coroutine. This creates a physical impact sensation that makes hits feel weighty and real rather than abstract health number changes.

The second effect is a red screen flash: a full-screen semi-transparent red UI Image overlay activates and its alpha fades from 0.35 to 0 over 0.4 seconds using a second coroutine with Mathf.Lerp(). This gives the player immediate visual confirmation that they were hit even if they did not see the enemy or spike that caused the damage.

Together these two effects run simultaneously and are triggered by a single TakeDamage() call in PlayerHealth.cs, making the implementation clean and centralized. The feature adds meaningful gameplay value by providing clear hit feedback and adding visual polish that distinguishes the game from a basic prototype.

6. Custom AI-Generated Assets

The following assets were created using AI image generation tools (Gemini) and are unique to this project:

Asset	Description	Used For
Heart Health Pack	Pixel art red heart icon, 32x32px, transparent background, retro platformer style	Health pack collectable - replaces default orange square
Spirit Orb Projectile	Glowing cyan pixel art orb, transparent background, retro platformer style	Player projectile fired on left click
Game Over Screen	Pixel art GAME OVER text with ghost decorations, dark background, red glowing text	Displayed when player health reaches 0
Level Complete Screen	Pixel art LEVEL COMPLETE text with stars and sparkles, gold glowing text	Displayed when player reaches the red flag finish line

7. Credits & External Assets

Asset / Tool	Source	License
Player, Enemy, Ground, Platform Tiles, Coins	Goldmetal Simple 2D Platformer Assets Pack	Free - Unity Asset Store Standard EULA
Heart, Spirit Orb, Game Over, Level Complete	AI-generated using Google Gemini	Custom - created for this project
TextMeshPro (Score & UI text)	Unity Built-in Package	Unity Companion License
Unity Engine	Unity Technologies - Unity 2022.3.62f3	Unity Personal License
Visual Studio 2022	Microsoft - IDE for C# scripting	Free Community Edition

8. Gameplay Demo Video Link

GD - Gameplay Demo Video Link